Redis Data Distribution Best Practices for Distributed Systems Archtectures

Redis Data Distribution Best Practices (TLDR)

Run **two separate Redis deployments** (or processes) under your installer: a **read-only product-owned Knowledge-Base (KB) instance** that you ship preloaded (dump.rdb) and a **separate runtime instance** for short/long memory that your product writes to and persists. Use application-level routing (two endpoints or a proxy) so KB updates never overwrite runtime data. This keeps installs fast and upgrades safe.

Why (concise rationale)

- An RDB snapshot is a point-in-time of *the entire dataset* the server holds — replacing dump.rdb or importing an RDB into an existing server will overwrite (or otherwise affect) that server's dataset. You cannot safely replace an RDB on a running server without risking user/runtime keys.

Redis "logical DBs" are all part of the same in-memory dataset and are saved together in the same RDB file — they do **not** give you separate persistence files per logical DB, so they do not solve the upgrade/overwrite risk. (Practically, separate processes = separate persistence.)

Shipping a prebuilt dump.rdb is a good way to get fast first installs, but you must isolate the KB dataset from the runtime dataset so future upgrades/restore actions do not wipe runtime data. Use an instance boundary for that isolation.

Recommended pattern (detailed)

1) Two-instance pattern (recommended)

- **KB Redis instance** (product-owned, read-mostly)
  - Separate redis-server process, separate data directory and dbfilename.
  - Ship a prebuilt `dump.rdb` in the installer and place it into KB instance `dir` before starting that server; redis will load it at startup. Use this instance as **read-only** from the application (or treat writes as controlled only by your upgrade scripts).
- Make it read-only either by design (app never writes) or by configuration (e.g., run as a replica of a master you control, or disable dangerous commands via `rename-command`, or enforce ACLs).
- **Runtime Redis instance** (user/runtime state)
  - Separate redis-server process, separate data directory, separate persistence strategy (AOF or frequent RDB + AOF hybrid) tuned for durability.
  - This instance must be the only place the application writes runtime memory and must be preserved across upgrades/backups.

How app accesses data: the application knows two endpoints (KB host:port, runtime host:port) and reads from KB and reads/writes to runtime. Alternatively put a small proxy in front that routes by key prefix.

Why this is best

- Completely isolates persistence lifecycles and crash/restore semantics.

- You can ship a large RDB for KB and never risk runtime state loss when reinstalling/upgrading KB.
- KB can be tuned for fast cold start and read throughput, runtime can be tuned for durability (AOF/fsync) and write latency.

---

2) Install / Upgrade flows (concrete)

First install (fast)

1. Installer copies prebuilt `dump.rdb` into the KB instance `dir` (the path from `CONFIG GET dir`) before starting KB server. Start KB server — it loads dataset quickly. (This is the typical import flow.)
2. Start runtime redis-server with an empty DB and configured persistence (AOF or RDB as appropriate).
3. App configured with two endpoints.

Normal upgrade that adds/updates KB docs

Option A — **Side-load + cutover (zero/low downtime)**

1. Build new KB `dump.rdb` for the new release (staging image).
2. Start an *auxiliary* KB instance on a different port with the new dump (or use a temporary container).
3. Warm it up and validate.
4. When ready, switch the app config / DNS / proxy to point KB traffic at the new KB instance (atomic cutover).
5. Decommission old KB process.

Option B — **Incremental update script** (suitable when deltas are small)

1. Produce an update script (or use `redis-cli --pipe`, `DUMP` / `RESTORE`, `MIGRATE` or a Redis client) that only writes the KB namespace (key prefix) on the KB instance.
2. Ship the update script as part of upgrade and run it against the KB instance. Because runtime lives in a different instance, no risk to runtime. See bulk loading patterns.

> Prefer Option A if many changes or you want reliable rollback; Option B is good if you can produce idempotent, small deltas.

Upgrading product + preserving runtime data

- Because runtime RDB/AOF sits in its own instance, you can upgrade product code without touching runtime persistence. If you must upgrade Redis itself, snapshot and backup runtime files first (or rely on replication) and then restore to new redis-server as described in Redis upgrade docs. Test RDB/AOF compatibility between versions before replacing files.

Persistence configuration recommendations (runtime)

- **AOF (Append Only File)** with `appendfsync` tuned to your durability needs (e.g., `everysec` is a common compromise) to avoid losing short-term memory during crashes. Consider AOF rewrite monitoring and disk sizing.

Optionally use hybrid RDB+AOF (`aof-use-rdb-preamble`) so recovery is fast but durable. Monitor AOF growth and rewrite times.

- Always have an automated backup and restore test for runtime files before any upgrade. Use replicas for high availability.

Bulk load / migration tools & commands (practical)

- **Ship dump.rdb** (KB): copy to KB `dir` then start server. (Import docs).

Bulk load into a running instance: `redis-cli --pipe` (fast client-side loader) or `MIGRATE` / `DUMP` + `RESTORE` for selective keys. See Redis bulk-loading guide.

Atomic cutover pattern: load new KB into separate process and swap endpoints. This avoids in-place wholesale restores that could overwrite other data.

Pitfalls & gotchas (must-watch)

1. **RDB file compatibility across Redis versions** — newer Redis generally reads older RDBs, but older Redis may reject RDBs produced by newer versions (RDB format/version differences). Test RDB file compatibility for installers and upgrades.

Modules & encoding — if KB dump was created with modules (RedisGraph, etc.), loading into a server without those modules will fail. Ensure KB instance runtime environment matches the environment used to produce the RDB.

- **Duplicate memory footprint** — two instances duplicate data in RAM if both run on same host. Ensure host sizing (RAM, CPU) is sized for the sum. Consider running on separate machines/containers if memory is constrained.
- **Operational complexity** — your installer now manages two Redis processes, ports, configs, backups; add health checks and clear logs/paths.
- **Security / ACLs** — separate ACLs for KB (read-only) vs runtime (read/write). Do not expose both on same network interface unless controlled.
- **Accidental imports overwrite data** — Redis import tools (RDB import, cloud import) will overwrite existing dataset; never run those against the runtime instance.

Example checklist for your installer (step-by-step)

Packaging

- Produce a KB `dump.rdb` (build step). Validate contents and Redis version used to create it.
- Package `dump.rdb` with the installer into a `kb/` directory and include a KB redis.conf tuned for read throughput.

Install

1. Create folders `data/kb` and `data/runtime` with correct ownership/permissions.
2. Copy `kb/dump.rdb` → `data/kb/dump.rdb`.
3. Start KB Redis process with `--dir data/kb --dbfilename dump.rdb` (or use the KB redis.conf). Validate keys count.
4. Start runtime Redis process with its own data dir and persistence config (AOF or RDB as selected).
5. Configure application with both endpoints and test behavior (reads from KB, writes to runtime).

Upgrade KB

- If small change: run idempotent key-level updates against KB instance (scripted `RESTORE` / `DEL` /sets).

- If large change: create new KB instance with new dump, validate, cutover by switching the KB endpoint in config/proxy.

Backups

- Backup runtime AOF/RDB regularly. Test restore to a new server before shipping upgrades.

References (key docs)

- Redis persistence overview (RDB/AOF): Redis docs.

Bulk loading patterns and guidance (Redis - The Real-time Data Platform bulk-loading).

Importing datasets / warning that import overwrites dataset (Redis Cloud / import docs).

RDB format/version notes (RDB encoded version, compatibility caveats).

Quick summary + recommendation

1. **Do not** rely on logical DBs to separate KB vs runtime persistence. Logical DBs are part of one RDB snapshot.

Do run two Redis instances (KB + runtime). Ship KB as a prebuilt `dump.rdb` for fast first install and keep runtime state isolated and persisted using AOF or an RDB/AOF hybrid.

For upgrades, prefer **side-load + cutover** (safe and rollback-friendly) or **incremental delta scripts** for small changes. Always test RDB compatibility and module parity between builds.

Installer flow + pseudoscripts (concrete, ready-to-adapt)

Below are concrete commands, example `redis.conf` fragments, systemd unit examples, and shell pseudocode you can drop into your installer. Paths and ports are illustrative — adjust to your packaging conventions.

Notes before you start:

• Replace `REDIS_BIN=/usr/local/bin/redis-server` and `REDIS_CLI=/usr/local/bin/redis-cli` as required.

• Use non-root user (e.g., `redis`) for runtime files. Ensure correct ownership.

• Test everything on the exact target OS images you ship to validate library/module compatibility.

Files / layout (Python example)

```
/opt/myproduct/
  kb/                   # packaged knowledge base files
    dump.rdb            # prebuilt KB snapshot
    redis-kb.conf
  runtime/
    redis-runtime.conf
  bin/
    start-all.sh
    stop-all.sh
    upgrade-kb.sh
    apply-kb-delta.sh
  service/              # optional systemd units
    redis-kb.service
    redis-runtime.service
```

Example redis.conf (KB instance — read-mostly)

Put this in `redis-kb.conf`:

```
bind 127.0.0.1
port 6379                   # KB port — choose a distinct port
dir /opt/myproduct/kb
dbfilename dump.rdb
save ""                      # disable periodic RDB saves if you only want the shipped file (optional)
appendonly no
# lock down dangerous commands if desired
rename-command FLUSHDB ""
rename-command FLUSHALL ""
# ACLs — optional (Redis >=6)
# user default on nopass ~* +@readonly
```

Comments:

- `save ""` prevents automatic RDB creation by this process if you want to ensure KB is exactly the shipped file. If you prefer to let KB persist runtime additions, remove `save ""`.
- Consider `rename-command` or ACLs to make KB effectively read-only.

Example redis.conf (Runtime instance — durable)

Put this in `redis-runtime.conf`:

```
bind 127.0.0.1
port 6380                   # runtime port
dir /opt/myproduct/runtime
dbfilename dump.rdb
appendonly yes
appendfsync everysec        # durable but performant compromise
aof-use-rdb-preamble yes    # hybrid AOF/RDB preamble for faster restarts
save 900 1 300 10 60 10000  # optional RDB snapshots; tune to your needs
```

Comments:

- AOF is recommended to protect runtime state. Tune `appendfsync` to `always` only if you need absolute zero-loss and can accept latency cost.

Systemd unit examples (optional)

`/etc/systemd/system/redis-kb.service`:

```
[Unit]
Description=Redis KB instance
After=network.target

[Service]
User=redis
ExecStart=/usr/local/bin/redis-server /opt/myproduct/kb/redis-kb.conf
Restart=on-failure

[Install]
WantedBy=multi-user.target

```

`/etc/systemd/system/redis-runtime.service` similar but points to runtime conf and uses different service name.

Installer pseudocode: first install (fast)

`start-all.sh` (pseudocode shell)

```
#!/bin/bash
set -e

REDIS_BIN=/usr/local/bin/redis-server
KB_DIR=/opt/myproduct/kb
RUNTIME_DIR=/opt/myproduct/runtime

# 1) create dirs
mkdir -p $KB_DIR $RUNTIME_DIR
chown -R redis:redis $KB_DIR $RUNTIME_DIR
chmod 700 $KB_DIR $RUNTIME_DIR

# 2) copy prebuilt KB dump into place (packaged with installer)
cp /installer_assets/dump.rdb $KB_DIR/dump.rdb
chown redis:redis $KB_DIR/dump.rdb

# 3) start KB (loads dump.rdb quickly)
sudo -u redis $REDIS_BIN /opt/myproduct/kb/redis-kb.conf &

# 4) start runtime (empty DB initially)
sudo -u redis $REDIS_BIN /opt/myproduct/runtime/redis-runtime.conf &
```

After start, verify:

```
# KB: count keys
/usr/local/bin/redis-cli -p 6379 DBSIZE
# Runtime: should be zero or minimal
/usr/local/bin/redis-cli -p 6380 DBSIZE
```

Creating the shipped `dump.rdb` (build pipeline)

On your build machine (same Redis version as target VM ideally):

```
# populate a staging redis instance with KB documents
redis-server --port 16379 --dir /tmp/kb_build --dbfilename dump.rdb --save ""
# load data by script / bulk loader into port 16379
# verify keys and format, then:
redis-cli -p 16379 SAVE
# copy dump for packaging
cp /tmp/kb_build/dump.rdb /path/to/installer_assets/dump.rdb
```

Important: create `dump.rdb` with the same Redis version (or one compatible with target version) and with same modules loaded (if KB uses modules).

Upgrade flows: side-load + cutover (recommended for large changes)

`upgrade-kb.sh` pseudocode:

```
#!/bin/bash
# 1) installer has new dump.rdb in /installer_assets/new-dump.rdb

# 2) start a temporary KB on alternate port 6371
TMP_DIR=/opt/myproduct/kb_tmp
mkdir -p $TMP_DIR
cp /installer_assets/new-dump.rdb $TMP_DIR/dump.rdb
chown redis:redis $TMP_DIR/dump.rdb

# start temp server with its own conf (port 6371)
redis-server --port 6371 --dir $TMP_DIR --dbfilename dump.rdb --save "" &

# 3) validate (key counts, health)
redis-cli -p 6371 DBSIZE
# optionally run test queries to validate content

```

```
# 4) atomic cutover: update app config or proxy to point KB endpoint to 6371
# e.g., update nginx/upstream or change service discovery entry

# 5) stop old KB and replace /opt/myproduct/kb with tmp for next upgrades if desired
```

Cutover ensures runtime instance never touched.

Upgrade flows: small incremental (delta) updates

If deltas are small, produce an idempotent script that modifies only KB keys and apply to KB instance.

Example `apply-kb-delta.sh` using `redis-cli --pipe` with an RDB-free approach:

```
# delta.txt contains a bulk Redis protocol stream, or use `redis-cli --pipe`
cat delta.txt | redis-cli -p 6379 --pipe
```

Or, use `DUMP` / `RESTORE` migration from a staging server:

```
# from staging: for key in $(redis-cli -p 16379 KEYS "kb:*"); do
#   redis-cli -p 16379 DUMP $key | ... RESTORE to production KB
# done
```

Make deltas idempotent (use `RESTORE ... REPLACE` or `SET` semantics).

Backup & restore runtime (scripts)

Backup runtime AOF:

```
# pause writes if you can (app-level), else copy atomically via rsync
systemctl stop myproduct-app
cp /opt/myproduct/runtime/appendonly.aof /backups/runtime-aof-$(date +%F).aof
systemctl start myproduct-app

```

Restore runtime to new server:

```
# stop runtime redis
systemctl stop redis-runtime
# copy backed-up aof/dump into runtime dir
cp /backups/runtime-aof-2026-01-01.aof /opt/myproduct/runtime/appendonly.aof
chown redis:redis /opt/myproduct/runtime/appendonly.aof
systemctl start redis-runtime

```

Always test restores on staging before applying in production.
